

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Program: BSc in Computer Science and Engineering

Course Code: CSE 3213

Assignment 3

Date of Submission: 19.09.2026

Submitted by,

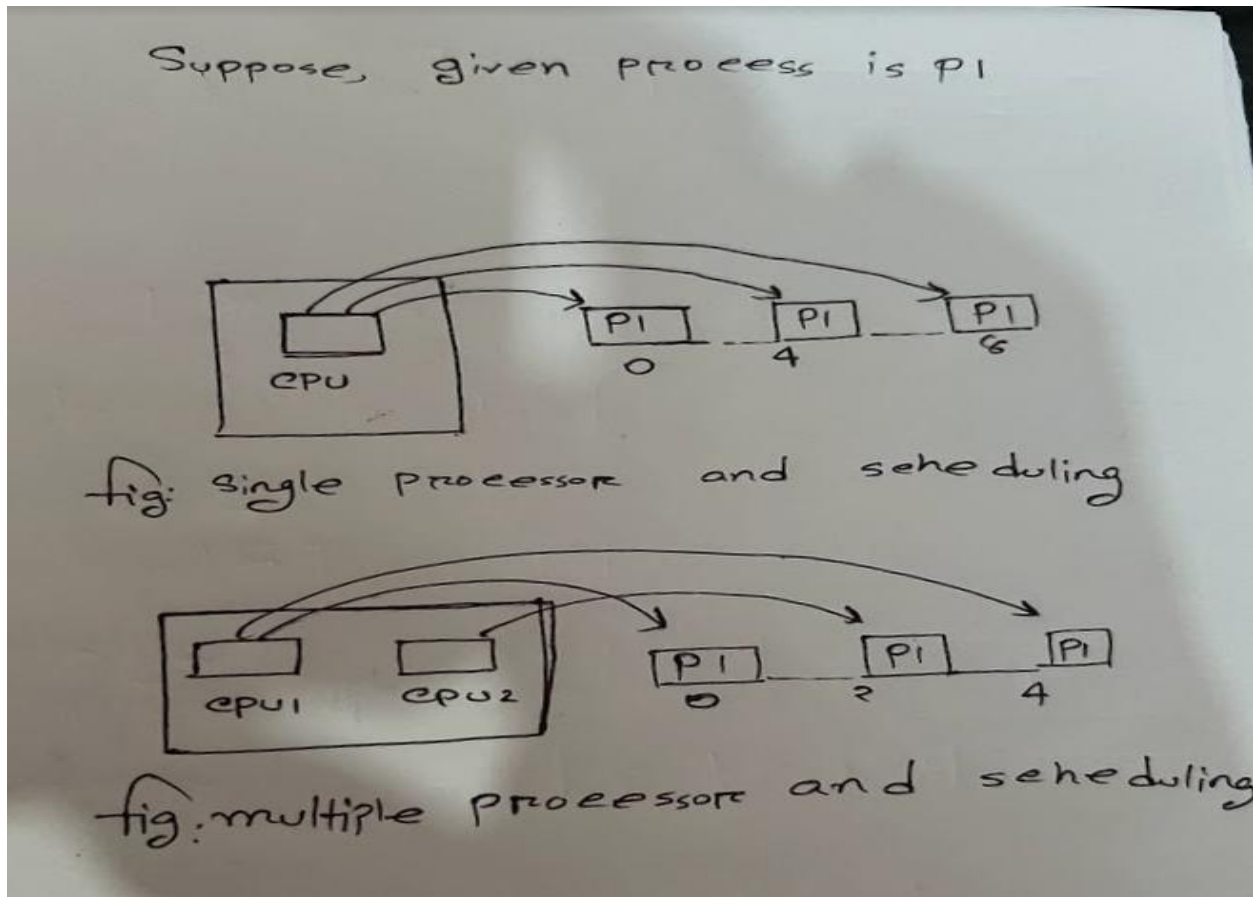
Name: Danial Hossain Dani

Student ID: 20230104058

Lab Section: B1

Multi-Processor Scheduling

In the previous chapters we studied scheduling in a system where there was only one processor. We learned how much CPU time each process get and when get the process. This was decided by different scheduling algorithms.



However, in case of multiprocessors or multicore computers, there are more than one processors. Suppose we have one processor and many processes. Now, any process might be allotted CPU time after periods like 0, 4, 8, and so forth. However, when we consider multiple processors, there are more than one processors that could run simultaneously. As such, any process would get CPU time after shorter durations. For instance, rather than obtaining CPU time after 0, 4, 8 periods, it would obtain CPU time after 0, 2, 4 periods with two processors running.

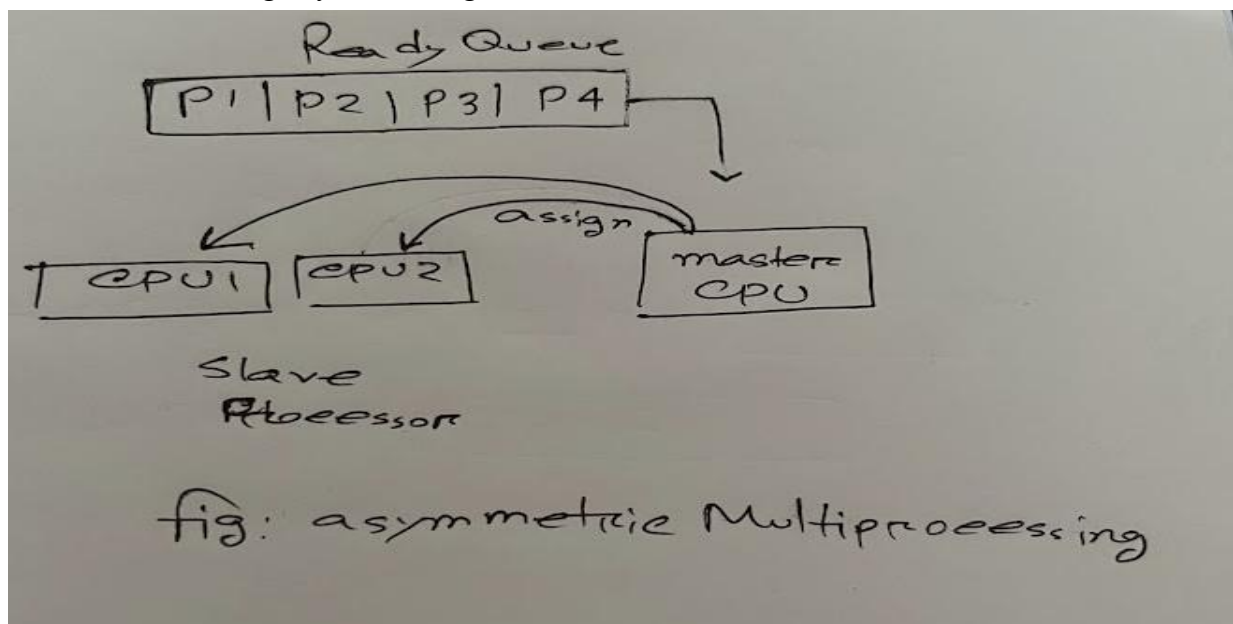
A system made up of multiple CPUs is able to run various processes simultaneously, which in turn presents the system with an important scheduling problem—deciding which process should be assigned to which processor. The method by which these decisions are reached is known as Multiprocessor Scheduling, and it aids in the distribution of processes among the available processors with the aim of ensuring that they work efficiently and that the system's overall performance is improved.

There are two major ways of multiprocessor scheduling:

1. Asymmetric Multiprocessing (AMP)
2. Symmetric Multiprocessing (SMP)

Asymmetric Multiprocessing(AMP)

Asymmetric Multiprocessing is a master processor and slave processors. The master processor is involved in the operating system scheduler operation. This processor will determine which process runs on which processor. The slave processors will just execute the processes assigned to them, without making any scheduling decisions.



Advantages:

Scheduling decisions will be made solely by the master processor and the system will be centralized.

Disadvantages:

The main role of the master process is task scheduling and it does not take part in executing normal application processes. Some process allocation may cause some idle processors.

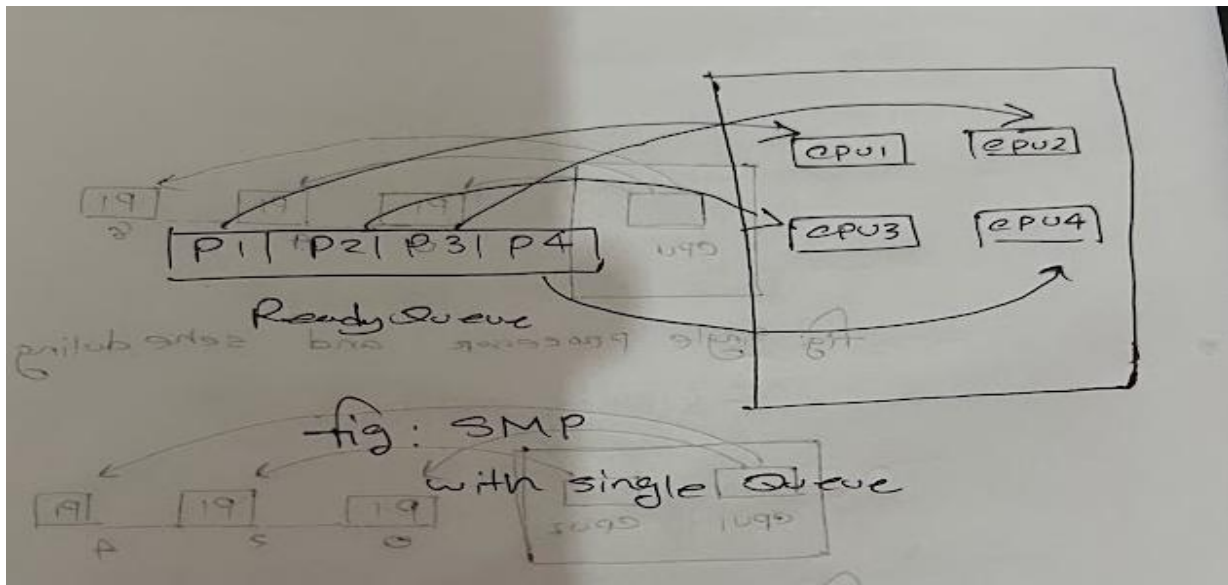
Symmetric Multiprocessing(SMP)

In symmetric multiprocessing, there is a scheduler associated with each CPU that determines which process it should execute; unlike in asymmetric multiprocessing, no CPU is in charge of scheduling all the processes.

The two main categories of SMP include:

1. SMP with Single Queue

Symmetric multiprocessing (SMP) systems use multiple CPUs that share the same memory. In an SMP setup with a single queue, all processors pull tasks from one central queue. This setup is simple and easy to manage. However, as the number of CPUs grows, the single queue can become a bottleneck. This happens because all processors compete for access to the queue. This competition can slow down the system and limit how much the workload can grow. The only ready queue present contains all the processes that are ready to run, and the CPUs select the processes to execute from the single ready queue which is used by all of them.



Advantages:

Each of the processes has the same chance of being carried out.

The CPU is used much more efficiently in this case.

Disadvantage:

Two or more CPUs might select the same process from the ready queue. Since it is impossible for multiple CPUs to carry out the same process at the same time, the processes chosen by them must be different.

In order to prevent the scenario in which two or more CPUs have access to the same process at the same time, a synchronization method such as semaphores can be used to lock the ready queue. For example, when CPU 1 is selecting a process from the ready queue, the queue can be locked by means of a semaphore.

2. SMP using multiple queues

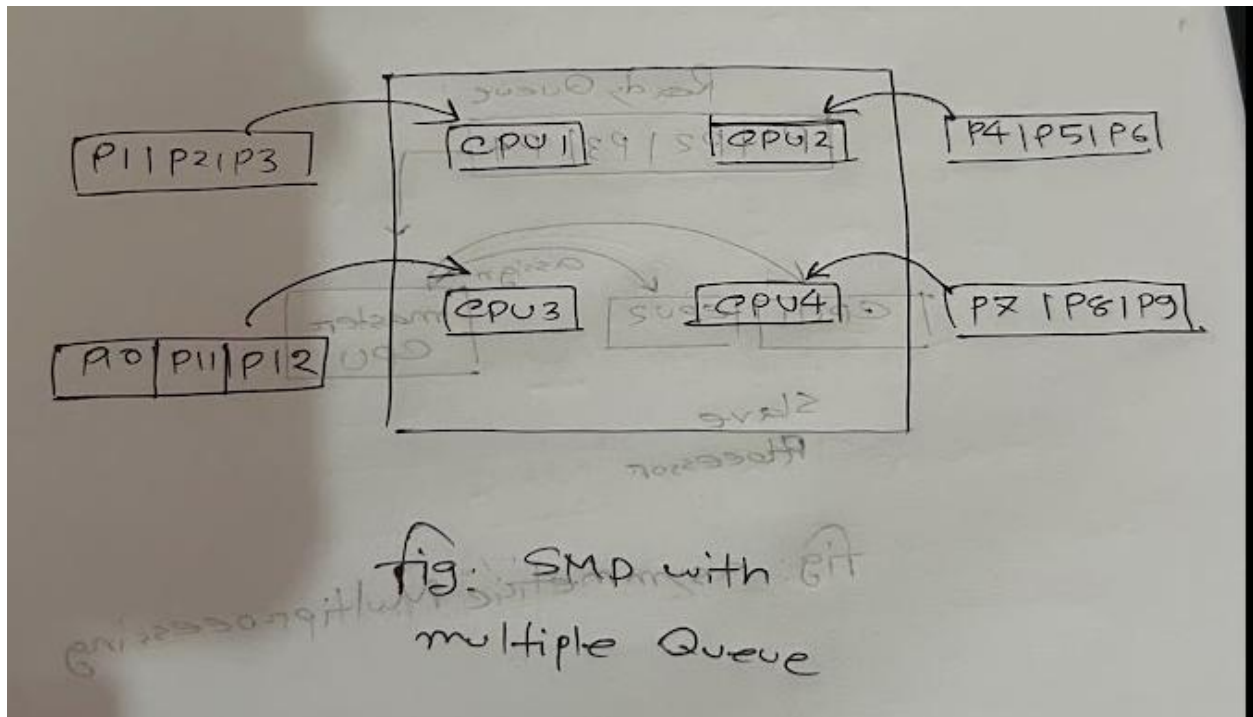
In this method, each CPU has its own ready queue and only selects and carries out processes from the one that has been allocated to it. This means that each CPU can look after its processes on its own, without having to access a shared queue.

Advantages:

There is no need for queue locking since each CPU has its own ready queue. It offers fast and scalable scheduling. Each central processing unit can look after its own processes.

Disadvantage:

A situation in which the load is unbalanced arises when some of the CPUs have a large number of processes in their queues while others have fewer or no processes to carry out. This causes some CPUs to stay idle while others are busy.



Load Balancing

In the SMP system, when there are multiple queues, each CPU has its own ready queue. There are instances when the processes in one queue finish more quickly than those in another queue. This causes one CPU to become idle while the others still have a large number of processes awaiting execution. A load imbalance thus occurs and results in inefficient use of the CPUs.

In order to deal with this issue, load balancing techniques are employed. The primary aim is to spread out the processes among the CPUs so that the workload is even and the CPUs can stay as busy as possible.

There are two main methods of load balancing:

1. Push Migration

In the case of push migration, a special system task regularly examines the amount of work being done by the CPUs. For instance, it might check the CPU load every 20 milliseconds. When a CPU is found to be heavily loaded, the system will transfer some of the processes away from that CPU and assign them to an idle CPU or one that has a lighter workload.

2. Pull Migration

In the case of pull migration, an idle CPU looks to see if any of the other CPUs have processes available. If it discovers a busy CPU that has a number of processes waiting, the idle CPU takes one or more of those processes for execution. This in turn helps to distribute the workload more evenly.

Cost of Process Migration:

Process migration can be costly since when a CPU is carrying out a process, part of that process's data may be kept in the CPU cache. If the process is then transferred to a different CPU, the required data might not be in the cache of that CPU. The new CPU will thus have to load the data into its cache once more. This takes additional time and can reduce overall system performance.

Processor Affinity

What processor affinity means is that a process should be assigned to a specific CPU for as long as possible so that it can make use of the data which is already held in that CPU's cache. This in turn leads to a reduction in the need for reloading the cache and results in better system performance.

There are two types of processor affinity:

1. Hard Affinity

Hard affinity is a rule which specifies that a process must run on a certain CPU or on a particular set of CPUs and it cannot be moved to a CPU that is outside this assigned set.

2. Soft Affinity

The operating system aims to keep a process running on the same CPU in the case of soft affinity, but it may move the process to a different CPU when needed, particularly in order to improve load balancing.

Cache Affinity

What cache affinity entails is that, if possible, the process should be kept running on the same CPU on which it was previously running so that it can make use of the data and instructions that are already held in that CPU's cache. This in turn reduces the number of cache misses and the need to reload data, thereby improving performance.

In simple terms, cache affinity refers to keeping a process near the data which is already stored in the CPU cache.

Affinity vs Load Balancing



There is usually a compromise involved in processor affinity and load balancing. If the processor affinity is high, processes will tend to stay on the same CPU more frequently, which can enhance cache performance but may make it harder to achieve load balancing. Conversely, if the processor affinity is low, the operating system has more scope to move processes between CPUs, which can improve load balancing but may result in more process migration and greater cache-related overhead.

Since processes are restricted by hard affinity, the operating system has very little capacity to transfer them between CPUs as a means of achieving load balancing; hence, in certain cases it may move a process even though it has a preference for a particular CPU in order to keep the workload better balanced.